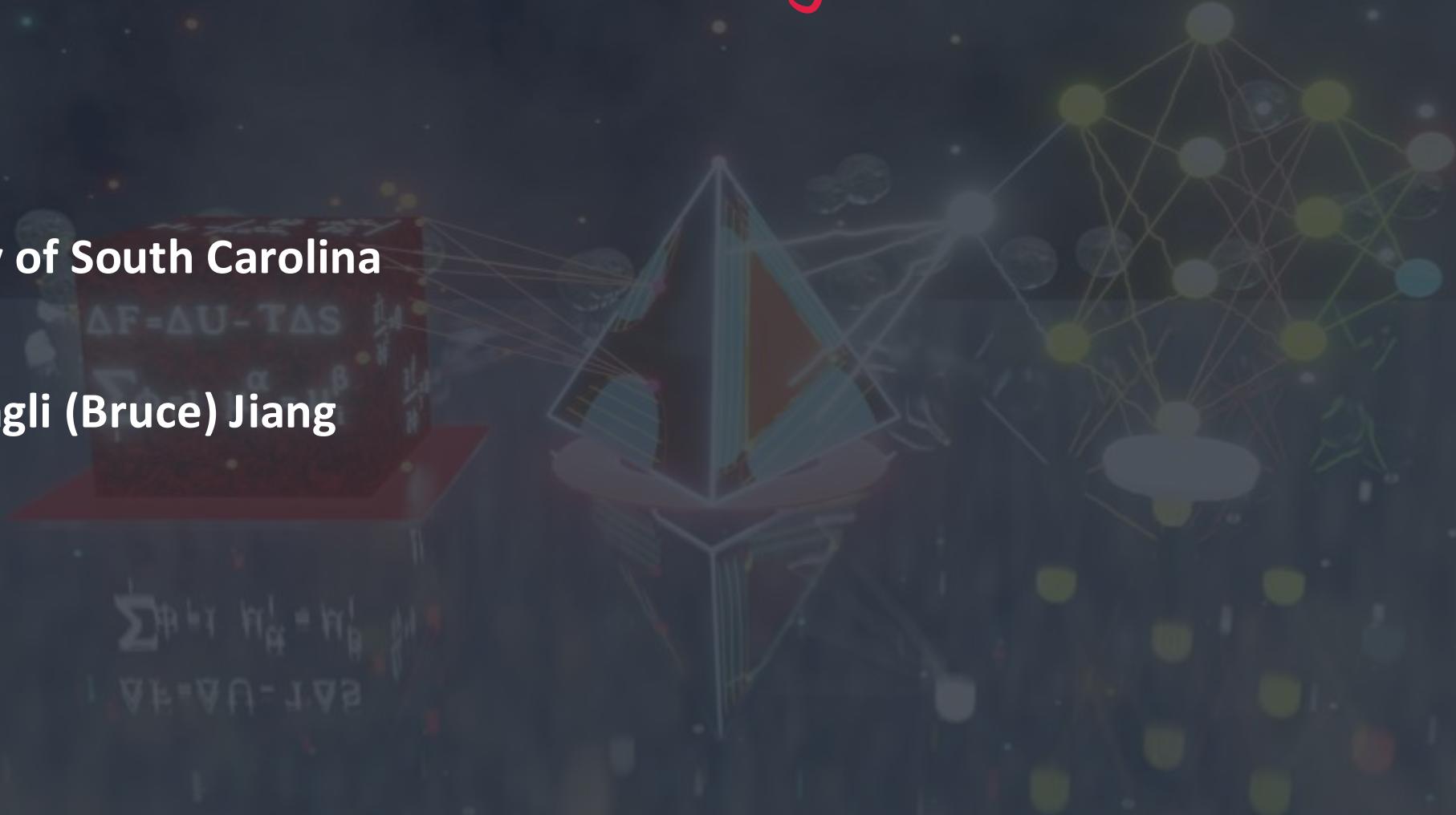


# Python Foundation and ~~Boring~~ Math Review I

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



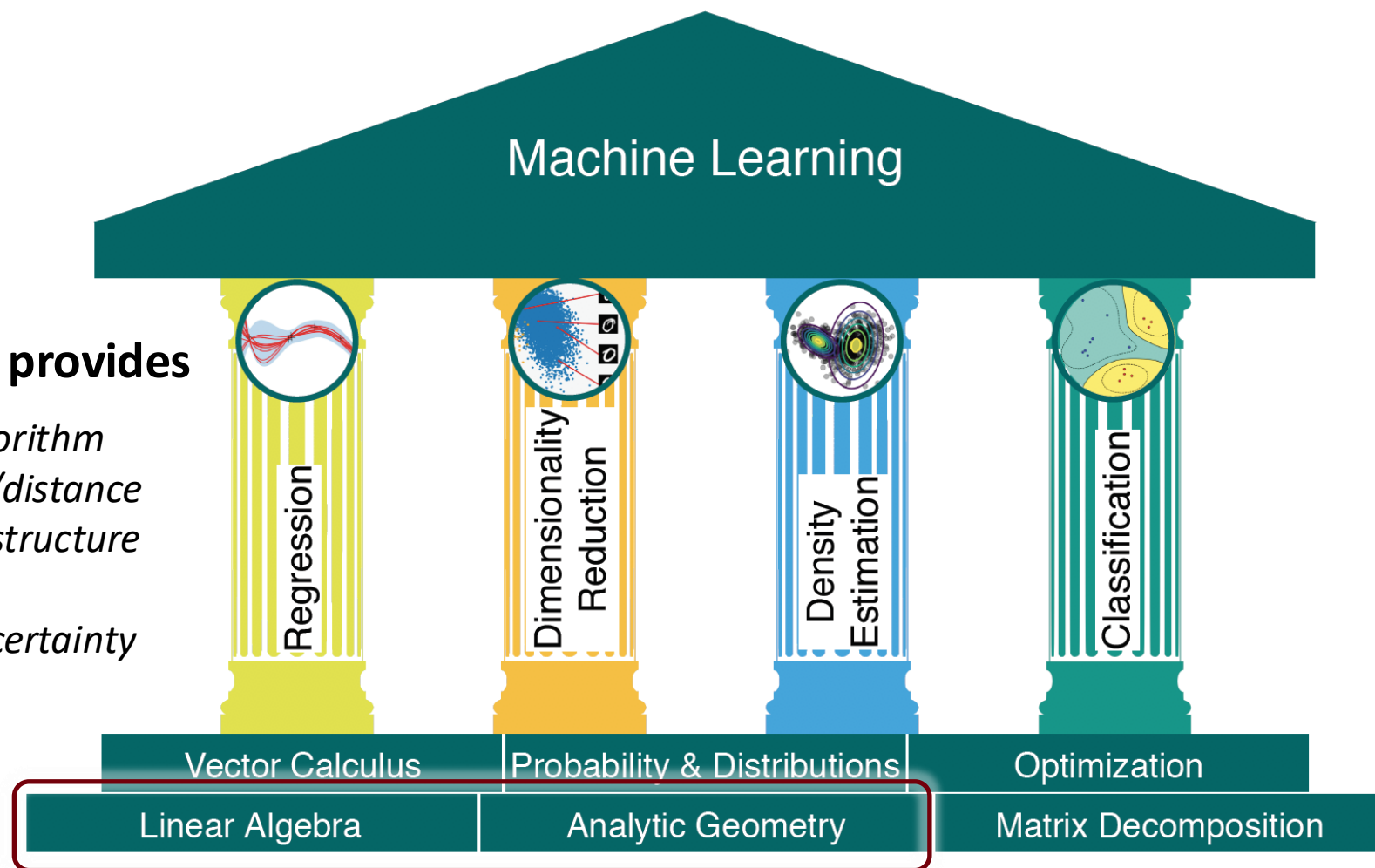
# Mathematics in Machine Learning

## Machine learning involves three connected tasks

- *Representing data*
- *Modeling data*
- *Learning patterns from data*

## What each mathematical foundation provides

- *Linear algebra -> represent data, ML algorithm*
- *Analytic geometry -> Measure similarity/distance*
- *Matrix decomposition -> Reveal simpler structure*
- *Vector calculus -> Compute gradients*
- *Probability & distribution -> Quantify uncertainty*
- *Optimization -> Fit model parameters*



**Start with linear algebra and analytic geometry**

# NumPy, Our Mathematical Partner

NumPy

Software

Sounds like

nuhm·pee

Slow

## NumPy

Article [Talk](#)

From Wikipedia, the free encyclopedia

**NumPy** (pronounced /nʌmpaɪ/ *NUM-py*) is a [library](#) for the [Python programming language](#), adding support for large, multi-dimensional [arrays](#) and



### Before NumPy

*In the 1990s, “**Numeric**” package was fast with small arrays (in fusion research).*



*Astronomers needed to process very large images from the Hubble Space Telescope -> created “**Numarray**” (slower for small arrays)*



*Unified both and created “**NumPy**”*

Travis Oliphant

2005



*Co-founded **Anaconda** with Peter Wang in 2012*

| List                                                  | NumPy                                                    |
|-------------------------------------------------------|----------------------------------------------------------|
| Store, append, remove, or mix general Python objects. | Mathematical operations on many <b>same-type</b> numbers |

# Essential Definitions: Vectors, Matrices, Tensors

$\mathbf{x}$  Vector: (denoted in ***bold lowercase***) elements of a **Vector Space**  $V$

$V$  Vector Space: a set of elements  $\mathcal{V}$  with the operations of addition and scalar multiplication, such that

$\mathcal{V} + \mathcal{V} \in \mathcal{V}$  a vector in the space plus another in the space yields another in the space

$\mathbb{R} \times \mathcal{V} \in \mathcal{V}$  a vector times a real number in the space yields a vector in the space

You are very familiar with  
Euclidean vectors in  $\mathbb{R}^3$

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

or perhaps even  $\mathbb{R}^n$   
(*tuples of  $n$  real numbers*)

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

## List <-> NumPy Array Conversion

```
import numpy as np
```

```
lst = [0.1, 0.2, 0.3, 0.4]
```

```
print(lst, type(lst))
```

```
arr = np.array(lst)
```

```
print(arr, type(arr))
```

```
lst2 = arr.tolist()
```

```
print(lst2, type(lst2))
```

✓ 0.0s

```
[0.1, 0.2, 0.3, 0.4] <class 'list'>
```

```
[0.1 0.2 0.3 0.4] <class 'numpy.ndarray'>
```

```
[0.1, 0.2, 0.3, 0.4] <class 'list'>
```

# Essential Definitions: Vectors, Matrices, Tensors

Matrix:

For  $m, n \in \mathbb{N}$

an  $(m,n)$  matrix is composed of an  $mn$ -tuple of elements arranged into  $m$  rows and  $n$  columns

**$A$**

(denoted in **BOLD  
UPPERCASE**)

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}, \quad a_{ij} \in \mathbb{R}$$

We can say that  $A \in \mathbb{R}^{m \times n}$

*(so can we say  $A$  lives in a vector space and is thus a vector? Sure!)*

# Essential Definitions: Vectors, Matrices, Tensors

Matrix:

For  $m, n \in \mathbb{N}$

an  $(m,n)$  matrix is composed of an  $mn$ -tuple of elements arranged into  $m$  rows and  $n$  columns

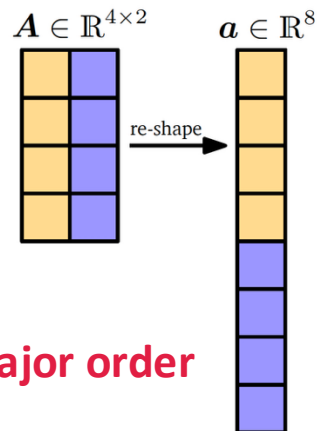
**$A$**   
(denoted in **BOLD  
UPPERCASE**)

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}, \quad a_{ij} \in \mathbb{R}$$

We can say that  $A \in \mathbb{R}^{m \times n}$

*Matrices can be represented as vectors by stacking columns vertically or stacking rows sequentially  
(and this is really how they would be represented on machine memory)*

$$A \in \mathbb{R}^{m \times n} \rightarrow a \in \mathbb{R}^{mn}$$



Column-major order

(Aside: Python  
stores elements of  
arrays in **ROW-  
MAJOR** order)

```
import numpy as np
arr = np.array([[1,3,5], [2,4,6]])
print(arr)
print("-"*20)
print(arr.flatten())
```

✓ 0.0s

```
[[1 3 5]
 [2 4 6]]
```

-----  
[1 3 5 2 4 6]

# Essential Definitions: Vectors, Matrices, Tensors

Tensor:

$\mathbf{T}, T_{ijk}...$

(denoted in **BOLD UPPERCASE** or explicit indices indicating rank)

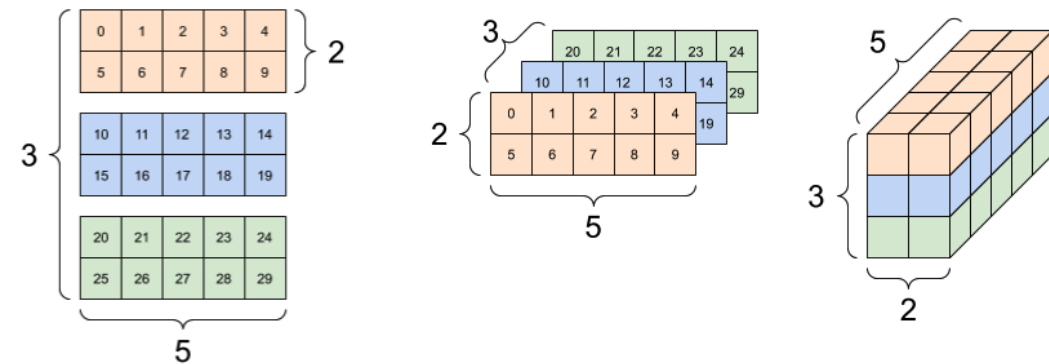
- Practically, tensors are objects that allow generalization of scalars, vectors, matrices, and higher-dimensional objects
- Tensors are defined by their **rank** (distinct from matrix rank)

*rank-0 tensor  $\rightarrow$  scalar*

*rank-1 tensor  $\rightarrow$  vector*

*rank-2 tensor  $\rightarrow$  matrix*

*rank-d tensor  $\rightarrow$  array with d axes*



**rank-3 tensor with shape 3x2x5**

*In python, NumPy arrays are effectively Tensor-like containers*

```
T = np.array([
    [0,1,2,3,4],
    [5,6,7,8,9],
    [10,11,12,13,14],
    [15,16,17,18,19],
    [20,21,22,23,24],
    [25,26,27,28,29]
])
print(T)
print("-"*20)
print(T[0])
```

```
[[[ 0  1  2  3  4]
  [ 5  6  7  8  9]]

 [[10 11 12 13 14]
  [15 16 17 18 19]]

 [[20 21 22 23 24]
  [25 26 27 28 29]]]

-----
[[0 1 2 3 4]
 [5 6 7 8 9]]
```

# Addition/Multiplication with Matrices

## Addition

$$\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{R}^{m \times n}$$

$$\mathbf{C} = \mathbf{A} + \mathbf{B}$$

$$c_{ij} = a_{ij} + b_{ij}$$

```
A = np.array([[1,2],[3,4]])
B = np.array([[8,7],[6,5]])
C = A + B
print(C)
```

✓ 0.0s

```
[[9 9]
 [9 9]]
```

## Scalar Multiplication

$$\mathbf{B} = \lambda \mathbf{A}, \lambda \in \mathbb{R}$$

$$b_{ij} = \lambda a_{ij}$$

## Multiplication

$$\mathbf{A} \in \mathbb{R}^{m \times n}$$

$$\mathbf{C} = \mathbf{A}\mathbf{B}$$

$$\mathbf{B} \in \mathbb{R}^{n \times k}$$

$$\mathbf{C} \in \mathbb{R}^{m \times k}$$

$$c_{ij} = \sum_{l=1}^n a_{il} b_{lj}$$

Shorthand Einstein notation:  
any index appearing twice in  
the same term implies  
summation over that index

$$c_{ij} = a_{il} b_{lj}$$

- matrix multiplication is not elementwise!

$$c_{ij} \neq a_{ij} b_{ij}$$

- Order matters!  $\mathbf{AB} \neq \mathbf{BA}$  in general

```
print(np.matmul(A,B))
print(np.dot(A,B))
print(np.einsum('il,lj->ij',A,B))
print(A@B)
```

✓ 0.0s

```
[[20 17]
 [48 41]]
[[20 17]
 [48 41]]
[[20 17]
 [48 41]]
[[20 17]
 [48 41]]
```

```
print(B@A)
```

✓ 0.0s

```
[[29 44]
 [21 32]]
```

# Addition/Multiplication with Matrices

## Addition

$$\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{R}^{m \times n}$$

$$\mathbf{C} = \mathbf{A} + \mathbf{B}$$

$$c_{ij} = a_{ij} + b_{ij}$$

```
A = np.array([[1,2],[3,4]])
B = np.array([[8,7],[6,5]])
C = A + B
print(C)
```

✓ 0.0s

```
[[9 9]
 [9 9]]
```

## Scalar Multiplication

$$\mathbf{B} = \lambda \mathbf{A}, \lambda \in \mathbb{R}$$

$$b_{ij} = \lambda a_{ij}$$

## Multiplication

$$\mathbf{A} \in \mathbb{R}^{m \times n}$$

$$\mathbf{C} = \mathbf{A}\mathbf{B}$$

$$\mathbf{B} \in \mathbb{R}^{n \times k}$$

$$\mathbf{C} \in \mathbb{R}^{m \times k}$$

$$c_{ij} = \sum_{l=1}^n a_{il} b_{lj}$$

Shorthand Einstein notation:  
any index appearing twice in  
the same term implies  
summation over that index

$$c_{ij} = a_{il} b_{lj}$$

```
print(np.matmul(A,B))
print(np.dot(A,B))
print(np.einsum('il,lj->ij',A,B))
print(A@B)
```

✓ 0.0s

```
[[20 17]
 [48 41]]
[[20 17]
 [48 41]]
[[20 17]
 [48 41]]
[[20 17]
 [48 41]]
```

## Elementwise multiplication (Hadamard product)

```
print(A*B)
print(np.multiply(A,B))
print(np.einsum('ij,ij->ij',A,B))
```

✓ 0.0s

```
[[ 8 14]
 [18 20]]
[[ 8 14]
 [18 20]]
[[ 8 14]
 [18 20]]
```

# Matrix Inverse and Transpose

Let  $\mathbf{A} \in \mathbb{R}^{n \times n}$  and  $\mathbf{B} \in \mathbb{R}^{n \times n}$

Suppose  $\mathbf{AB} = \mathbf{I}_n = \mathbf{BA}$

Then  $\mathbf{B} = \mathbf{A}^{-1}$  is the inverse of  $\mathbf{A}$

```
print(A)
B = np.linalg.inv(A)
C = np.linalg.inv(B)
print(B)
print(C)
```

✓ 0.0s

```
[[1 2]
 [3 4]]
[[-2.  1.]
 [ 1.5 -0.5]]
[[1. 2.]
 [3. 4.]]
```

Let  $\mathbf{A} \in \mathbb{R}^{n \times m}$  and  $\mathbf{B} \in \mathbb{R}^{m \times n}$

Suppose  $b_{ij} = a_{ji}$

Then  $\mathbf{B} = \mathbf{A}^T$  is the transpose of  $\mathbf{A}$

```
print(A.transpose())
```

✓ 0.0s

```
[[1 3]
 [2 4]]
```

```
print(A.T)
```

✓ 0.0s

```
[[1 3]
 [2 4]]
```

*Let's go practice in Python to convince ourselves of the following useful properties:*

$$\begin{aligned}\mathbf{AA}^{-1} &= \mathbf{I} = \mathbf{A}^{-1}\mathbf{A} \\ (\mathbf{AB})^{-1} &= \mathbf{B}^{-1}\mathbf{A}^{-1} \\ (\mathbf{A} + \mathbf{B})^{-1} &\neq \mathbf{A}^{-1} + \mathbf{B}^{-1} \\ (\mathbf{A}^\top)^\top &= \mathbf{A} \\ (\mathbf{A} + \mathbf{B})^\top &= \mathbf{A}^\top + \mathbf{B}^\top \\ (\mathbf{AB})^\top &= \mathbf{B}^\top \mathbf{A}^\top\end{aligned}$$

# Matrix Method for Solving a Linear system

We can generally use matrices to solve linear systems expressed as  $Ax = b$

... if  $A$  is square and *invertible*, then simply

*(approximate) matrix inversion and factorization are common mathematical procedures that underlie the formulation of many ML algorithms (e.g., linear+ridge regression, Gaussian processes, optimizations/updating)*

```
A = np.array([[1,2],[3,4]])  
b = np.array([6,7])  
x = np.linalg.solve(A, b)
```

```
print(x)  
print(np.allclose(A@x, b))
```

✓ 0.0s

[-5. 5.5]

True

$$x = A^{-1}b$$

*Not full rank!*

```
A = np.array([[1, 2],  
              |  |  |  |  [2, 4]])  
b = np.array([3, 6])
```

```
✦✦  
x = np.linalg.solve(A, b)
```

LinAlgError: Singular matrix

# Rank and Invertibility

In a matrix  $\mathbf{A}$ , the number of linearly independent columns equals the number of linearly independent rows, and we call this the **Rank**

*Two important properties:*

$$\forall \mathbf{A} \in \mathbb{R}^{m \times n} \text{ and } \mathbf{b} \in \mathbb{R}^m, \\ \exists \mathbf{x} \text{ s.t. } \mathbf{Ax} = \mathbf{b}, \text{ iff } \text{rank}(\mathbf{A}) = \text{rank}(\mathbf{A}|\mathbf{b})$$

*“A solution exists only if  $\mathbf{b}$  lies in the column space of  $\mathbf{A}$ ”*

$$\forall \mathbf{A} \in \mathbb{R}^{n \times n}, \exists \mathbf{A}^{-1} \text{ iff } \text{rank}(\mathbf{A}) = n$$

*“Invertible square matrices (regular) possess linearly independent columns”*

```
A = np.array([[1,2],[3,4]])
print("Rank of A:", np.linalg.matrix_rank(A))
B = np.array([[1,2],[2,4]])
print("Rank of B:", np.linalg.matrix_rank(B))
```

✓ 0.0s

Rank of A: 2

Rank of B: 1

*How can we tell what the rank is for a matrix?*

# A Chemically Motivated Example

Consider a simplified reaction network for carbon gasification and syngas chemistry.



- R1:  $\text{C} + \text{H}_2\text{O} \rightleftharpoons \text{CO} + \text{H}_2$
- R2:  $\text{C} + 2\text{H}_2\text{O} \rightleftharpoons \text{CO}_2 + 2\text{H}_2$
- R3:  $\text{C} + 2\text{H}_2 \rightleftharpoons \text{CH}_4$
- R4:  $\text{C} + \text{CO}_2 \rightleftharpoons 2\text{CO}$
- R5:  $\text{CO} + \text{H}_2\text{O} \rightleftharpoons \text{CO}_2 + \text{H}_2$

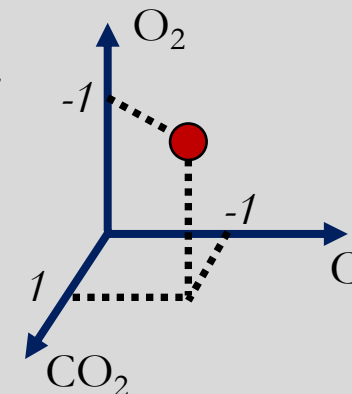
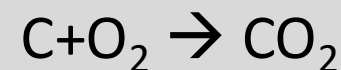
For reaction 1

- $\text{C} + \text{H}_2\text{O} \rightleftharpoons \text{CO} + \text{H}_2$

$$\nu_1 = [-1 \quad -1 \quad 1 \quad 1 \quad 0 \quad 0]$$

Species order =  $[\text{C}, \text{H}_2\text{O}, \text{CO}, \text{H}_2, \text{CO}_2, \text{CH}_4]$

We can represent reactions as linear combinations of molecules, with molecules or fragments representing a basis in chemical space



**How many independent species-change patterns are contained in these five reactions?**

$$\mathbf{A} = \begin{bmatrix} -1 & -1 & 1 & 1 & 0 & 0 \\ -1 & -2 & 0 & 2 & 1 & 0 \\ -1 & 0 & 0 & -2 & 0 & 1 \\ -1 & 0 & 2 & 0 & -1 & 0 \\ 0 & -1 & -1 & 1 & 1 & 0 \end{bmatrix}$$

# A Chemically Motivated Example

Consider a simplified reaction network for carbon gasification and syngas chemistry.



- R1:  $\text{C} + \text{H}_2\text{O} \rightleftharpoons \text{CO} + \text{H}_2$
- R2:  $\text{C} + 2\text{H}_2\text{O} \rightleftharpoons \text{CO}_2 + 2\text{H}_2$
- R3:  $\text{C} + 2\text{H}_2 \rightleftharpoons \text{CH}_4$
- R4:  $\text{C} + \text{CO}_2 \rightleftharpoons 2\text{CO}$
- R5:  $\text{CO} + \text{H}_2\text{O} \rightleftharpoons \text{CO}_2 + \text{H}_2$

For reaction 1

- $\text{C} + \text{H}_2\text{O} \rightleftharpoons \text{CO} + \text{H}_2$

$$\boldsymbol{\nu}_1 = [-1 \quad -1 \quad 1 \quad 1 \quad 0 \quad 0]$$

Species order =  $[\text{C}, \text{H}_2\text{O}, \text{CO}, \text{H}_2, \text{CO}_2, \text{CH}_4]$

```
print(A)
print(np.linalg.matrix_rank(A))
```

✓ 0.0s

```
[[ -1  -1   1   1   0   0]
 [ -1  -2   0   2   1   0]
 [ -1   0   0  -2   0   1]
 [ -1   0   2   0  -1   0]
 [  0  -1  -1   1   1   0]]
3
```

- R1:  $\text{C} + \text{H}_2\text{O} \rightleftharpoons \text{CO} + \text{H}_2$
- R3:  $\text{C} + 2\text{H}_2 \rightleftharpoons \text{CH}_4$
- R5:  $\text{CO} + \text{H}_2\text{O} \rightleftharpoons \text{CO}_2 + \text{H}_2$

**5 reactions, 3 independent stoichiometric directions**

$$\boldsymbol{\nu}_2 = \boldsymbol{\nu}_1 + \boldsymbol{\nu}_5, \quad \boldsymbol{\nu}_4 = \boldsymbol{\nu}_1 - \boldsymbol{\nu}_5$$

Any net composition change generated by the five reactions can be expressed using three independent stoichiometric directions.

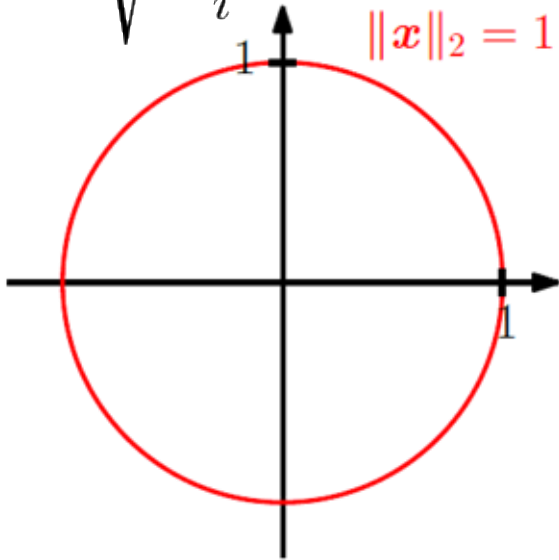
$$\Delta \mathbf{n} = \eta_1 \boldsymbol{\nu}_1 + \eta_3 \boldsymbol{\nu}_3 + \eta_5 \boldsymbol{\nu}_5$$

# Norms, Inner Products, and Distances

- A **norm** measures the length of a vector.
- A **distance** measures the separation between two vectors.

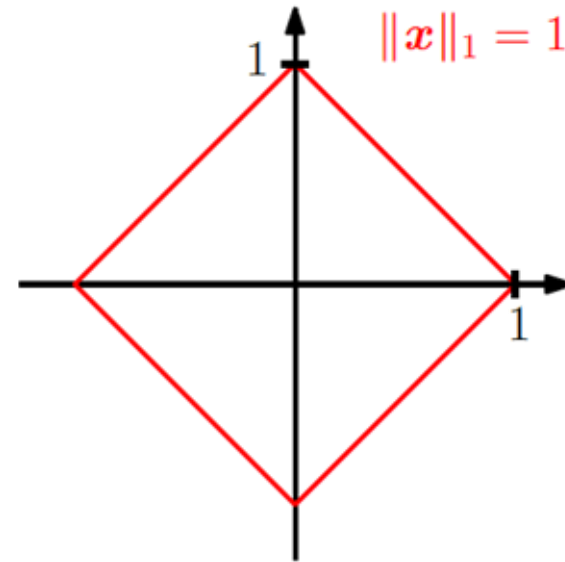
**Euclidean norm** (or  $l_2$ -norm)

$$\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2} = \sqrt{\mathbf{x}^T \mathbf{x}}$$



**Manhattan norm** (or  $l_1$ -norm)

$$\|\mathbf{x}\|_1 = \sum_i |x_i|$$



$$d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$$

# Norms, Inner Products, and Distances

- A **norm** measures the length of a vector.
- A **distance** measures the separation between two vectors.

**Euclidean norm** (or  $l_2$ -norm)

$$\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2} = \sqrt{\mathbf{x}^T \mathbf{x}}$$

```
x = np.array([3, 4])  
  
l1 = np.linalg.norm(x, ord=1)  
l2 = np.linalg.norm(x, ord=2)  
  
print(l1)  
print(l2)
```

✓ 0.0s

7.0

5.0

**Manhattan norm** (or  $l_1$ -norm)

$$\|\mathbf{x}\|_1 = \sum_i |x_i|$$

Example: Two models predict the outlet composition of [CO, H<sub>2</sub>, CO<sub>2</sub>, CH<sub>4</sub>]. The prediction-error vectors are

$$\mathbf{e}_A = \begin{bmatrix} 4 & 0 & 0 & 0 \end{bmatrix}$$

$$\mathbf{e}_B = \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix}$$

**Which model has the larger error?**

# Norms, Inner Products, and Distances

An inner product that you know:

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^T \mathbf{y} = \sum_i x_i y_i$$

**Every coordinate contributes equally.**

An example of one you probably don't (matrix-weighted norm):

$$\mathbf{A} = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}$$

$$\langle \mathbf{x}, \mathbf{y} \rangle_{\mathbf{A}} = \mathbf{x}^T \mathbf{A} \mathbf{y}$$

$$\mathbf{A} = \mathbf{A}^T$$

$$\mathbf{x}^T \mathbf{A} \mathbf{x} = (x_1 - x_2)^2 + x_2^2 > 0 \quad \text{for } \mathbf{x} \neq \mathbf{0}$$

$$\|\mathbf{x}\|_{\mathbf{A}} = \sqrt{\mathbf{x}^T \mathbf{A} \mathbf{x}}$$

**Symmetry makes the comparison independent of order. Positive definiteness guarantees a positive length for every nonzero vector.**

# Norms, Inner Products, and Distances

## Exercise:

*Compute the distance metric between the two vectors*

$$\mathbf{v}_1 = [3, 2]^T$$

$$\mathbf{v}_2 = [2, 1]^T$$

*using*

***dot product***

*vs.*

$$\mathbf{A} = \begin{bmatrix} 1 & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{bmatrix}$$

$$\boldsymbol{\delta} = \mathbf{v}_1 - \mathbf{v}_2 = \begin{bmatrix} 3 \\ 2 \end{bmatrix} - \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad d_2(\mathbf{v}_1, \mathbf{v}_2) = \sqrt{\boldsymbol{\delta}^T \boldsymbol{\delta}} = \sqrt{\begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix}} = \sqrt{2}$$

$$d_{\mathbf{A}}(\mathbf{v}_1, \mathbf{v}_2) = \sqrt{\boldsymbol{\delta}^T \mathbf{A} \boldsymbol{\delta}} = \sqrt{\begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix}} = 1$$

# Norms, Inner Products, and Distances

```
from scipy.spatial.distance import euclidean, mahalanobis
```

```
v1 = np.array([3, 2])
```

```
v2 = np.array([2, 1])
```

```
A = np.array([
    [1, -0.5],
    [-0.5, 1]
])
```

```
d2 = euclidean(v1, v2)
```

```
dA = mahalanobis(v1, v2, A)
```

```
print(d2)
```

```
print(dA)
```

✓ 10.2s

1.4142135623730951

1.0

A: Aerosols; Environmental and Atmospheric Chemistry;  
Astrochemistry | January 15, 2026

Open Access

## Fast and Interpretable Machine Learning Modeling of Atmospheric Molecular Clusters

Fast and Interpretable Machine Learning Modeling of Atmospheric Molecular Clusters

[Lauri Seppäläinen \\*](#); [Jakub Kubečka](#); [Jonas Elm](#); [Kai R. Puolamäki](#)

+ [Author & Article Information](#)

*J. Phys. Chem. A* (2026) 130 (4): 902–913.

<https://doi.org/10.1021/acs.jpca.5c06950>

Article history 

Metric learning learns **A** from data so that relevant samples become close and irrelevant samples become separated.

Euclidean closeness between molecular descriptors does not necessarily mean chemical relevance!

# Norms, Inner Products, and Distances

Any *inner product* also defines a *norm*:

$$\|x\| := \sqrt{\langle x, x \rangle}$$

The *distance (metric)* between two vectors is

$$d(x, y) := \|x - y\| = \sqrt{\langle x - y, x - y \rangle}$$

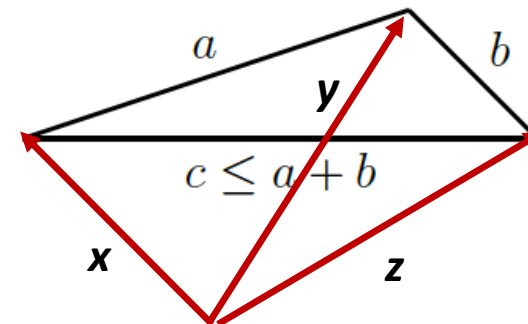
Cauchy-Schwarz Inequality

$$|\langle x, y \rangle| \leq \|x\| \|y\|$$

Triangle Inequality

$$d(x, z) \leq d(x, y) + d(y, z)$$

What inner product defines the Manhattan norm?

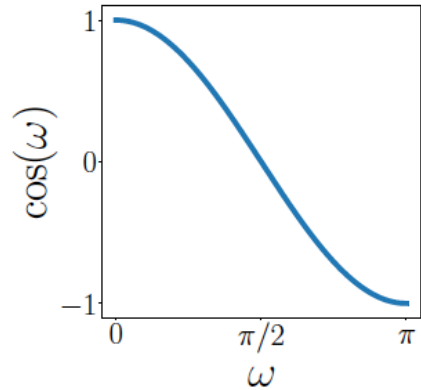


# Norms, Inner Products, and Distances

Inner products also enable definition of an angle between vectors

*Following Cauchy-Schwarz inequality,*

$$-1 \leq \frac{\langle x, y \rangle}{\|x\| \|y\|} \leq 1$$



there is a unique  
mapping to an angle  $\omega$   
on the interval  $[0,\pi]$

$$\cos \omega = \frac{\langle x, y \rangle}{\|x\| \|y\|}$$

***orthogonality***

$$\langle x, y \rangle = 0$$

```
import numpy as np
from scipy.spatial.distance import cosine
```

```
x = np.array([1, 0])
y = np.array([1, 1])
```

```
d_cos = cosine(x, y)
```

```
print(d_cos)
```

```
angle = np.degrees(np.arccos(1 - d_cos))
```

```
print(angle)
```

✓ 0.0s

0.29289321881345254

45.000000000000001

***cosine distance***

$$d_{\cos}(\mathbf{x}, \mathbf{y}) = 1 - \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2} = 1 - \cos \omega$$

# Interactive Python Time

02\_python\_foundation\_math.ipynb